

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Observation Task No.	6
Student Name	B.JATIN
Roll No.	A24126552070
Date	21-09-2026

1. TITLE

Node.js Built-in Modules: os, path and fs

2. AIM

To understand Node.js modules and demonstrate the purpose of the built-in os, path, and fs modules.

3. OBJECTIVES

1. Understand modularity in Node.js.
2. Use the os module for operating-system information.
3. Use the path module for safe file-path operations.
4. Use the fs module for file creation and reading.
5. Combine built-in modules in a Node.js program.

4. THEORY

Node.js modules provide reusable functionality. The os module exposes operating-system information, path provides utilities for handling file and directory paths, and fs provides file-system operations such as writing, reading, and deleting files.

5. ALGORITHM / PROCEDURE

1. Import os, path and fs using require().
2. Display platform, architecture and hostname using os.
3. Build a file path using path.join().
4. Extract the file name and extension using path.basename() and path.extname().
5. Create and write a file using fs.writeFileSync().
6. Read the file using fs.readFileSync().
7. Remove the temporary file using fs.unlinkSync().

6. PROGRAM

Source file: node_modules_demo.js

```
// Task 6: Node.js built-in modules - os, path and fs

const os = require("os");
const path = require("path");
const fs = require("fs");

console.log("NODE.JS BUILT-IN MODULES");

// os module: provides operating-system information.
console.log("\nOS MODULE");
console.log("Platform:", os.platform());
console.log("Architecture:", os.arch());
console.log("Hostname:", os.hostname());

// path module: works with file and directory paths.
console.log("\nPATH MODULE");
const filePath = path.join(__dirname, "sample.txt");
console.log("Joined path:", filePath);
console.log("File name:", path.basename(filePath));
console.log("Extension:", path.extname(filePath));

// fs module: creates, writes and reads files.
console.log("\nFS MODULE");
fs.writeFileSync(filePath, "Hello from the Node.js File System module.");
const content = fs.readFileSync(filePath, "utf8");
console.log("File content:", content);

// Clean up the demonstration file after reading it.
fs.unlinkSync(filePath);
console.log("Temporary file removed successfully.");
```

7. CODE / SYNTAX EXPLANATION

Code / Syntax	Explanation
require('os')	Loads the built-in operating-system module.
require('path')	Loads path utilities.
require('fs')	Loads file-system functions.
os.platform()	Returns the operating-system platform.
path.join()	Combines path segments.
path.basename()	Returns the final file name.
path.extname()	Returns the file extension.
fs.writeFileSync()	Writes data to a file synchronously.
fs.readFileSync()	Reads file contents synchronously.
fs.unlinkSync()	Deletes a file synchronously.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Status
TC-01	Run program	OS information displayed	Pass
TC-02	Generate path	Path and extension displayed	Pass
TC-03	Write sample.txt	File created with content	Pass
TC-04	Read sample.txt	Content displayed	Pass
TC-05	Finish execution	Temporary file removed	Pass

9. OUTPUT

Execution output captured from the local test run.

Task 6 - Execution Output

NODE.JS BUILT-IN MODULES

OS MODULE

Platform: linux
Architecture: x64
Hostname: localhost

PATH MODULE

Joined path: /mnt/data/PBL_Tasks_5_8/task6/sample.txt
File name: sample.txt
Extension: .txt

FS MODULE

File content: Hello from the Node.js File System module.
Temporary file removed successfully.

Output summary:

NODE.JS BUILT-IN MODULES

OS MODULE

Platform: linux
Architecture: x64

Hostname: localhost

PATH MODULE

Joined path: /mnt/data/PBL_Tasks_5_8/task6/sample.txt

File name: sample.txt

10. RESULT

Thus, the os, path and fs modules were successfully demonstrated for operating-system information, path handling and file management.